

Kentia Engineering

Calibration Manager

Documentación Técnica

Versión 1.0 · Mayo 2026

Uso interno — Equipo de desarrollo





Índice

1	Descripción general del sistema	3
2	Stack tecnológico y arquitectura	4
3	Estructura del repositorio	5
4	Configuración del entorno de desarrollo	6
5	Variables de entorno	7
6	Arranque en local	8
7	Modelos de datos — MongoDB	9
8	API REST — Referencia de endpoints	11
9	Autenticación JWT y sistema de roles	14
10	Build y empaquetado del instalador .exe	16
11	Bugs conocidos y sus soluciones	18
12	Registro de cambios (Changelog)	19



1 Descripción general del sistema

HERKO Calibration Manager es una aplicación de escritorio para Windows desarrollada por Kentia Engineering que gestiona el ciclo de vida completo de las calibraciones de software embebido en ECUs. Cubre desde la importación de ficheros A2L hasta la aprobación regulatoria, trazabilidad completa mediante audit log y asignación del software publicado a vehículos por VIN.

Módulos funcionales implementados

- **Software Releases:** registro de versiones de software ECU con ficheros A2L, DBC y DTC.
- **Datasets:** conjuntos de parámetros con ciclo de vida EDIT → RELEASED.
- **Labels/Parámetros:** gestión individual con niveles EMPTY → DOCUMENTED.
- **Review Center:** flujo de aprobación multi-rol independiente.
- **Release Center:** promoción y publicación de datasets.
- **Vehicle Assignment:** asignación de software RELEASED a VINs.
- **Merge de calibraciones:** overlay_wins / base_wins / manual con preview de conflictos.
- **Export DCM:** exportación DAMOS 2.0 filtrable por scope, maturity y filename.
- **Audit Log:** registro completo de todos los cambios.
- **Admin Panel:** gestión de usuarios y roles (exclusivo DM_Administrator).
- **Role switcher:** cambio de rol activo sin cerrar sesión.
- **Visualization:** módulo de visualización 3D / heatmaps (carpeta visualization/).

2 Stack tecnológico y arquitectura

Capa	Componente	Tecnología	Notas
Ventana nativa	Electron	Electron v28+	Envuelve el frontend en ventana Windows nativa. Sin chrome de navegador.
Frontend	React SPA	React 18 + React Router v7	Build con CRACO. UI: Radix UI + Tailwind CSS + shadcn/ui. Forms: RHF + Zod.
HTTP Client	Axios	Axios	Interceptor automático para inyectar token JWT en Authorization header.
Backend	API REST	FastAPI + Python 3.11 + Uvicorn	Sirve también el build estático de React en producción via StaticFiles.
Auth	JWT + bcrypt	PyJWT + bcrypt	Tokens HS256. Hash bcrypt para contraseñas. Roles en payload JWT.
Base de datos	MongoDB	Motor (async driver)	MongoDB Atlas (cloud). Driver asíncrono. Base de datos: calibrationdb.
Instalador	Inno Setup	ISCC.exe	Genera CalibrationManager_Setup.exe con asistente de instalación Windows.

■ NOTA TÉCNICA

Flujo de datos: Electron arranca CalibrationManagerServer.exe como proceso hijo. Polling a localhost:8000 cada 500ms hasta que responde (timeout 30s). BrowserWindow carga <http://localhost:8000>. React llama a /api/* via Axios. FastAPI procesa y consulta MongoDB Atlas. Al cerrar la ventana, Electron mata el servidor.

3 Estructura del repositorio

```
kentia_calibration/
├── backend/
│   ├── .env Variables de entorno (MONGO_URL, JWT_SECRET...)
│   ├── server.py API FastAPI principal (~950 lineas)
│   ├── models.py Modelos Pydantic y tipos de datos
│   ├── auth_utils.py JWT encode/decode, bcrypt, extraccion de token
│   ├── seed.py Seed completo (usuarios, releases, datasets, labels)
│   ├── seed_users_simple.py Seed solo de usuarios (util para recuperar login)
│   ├── check_db.py Diagnostico de conexion a MongoDB Atlas
│   ├── requirements.txt Dependencias Python
│   ├── tests/
│   └── test_herko_api.py Tests de integracion de la API
├── frontend/
│   ├── .env REACT_APP_BACKEND_URL
│   ├── package.json Dependencias Node.js
│   ├── craco.config.js Configuracion personalizada de build
│   ├── src/
│   │   ├── App.js Router principal (React Router v7)
│   │   └── lib/
│   │       ├── api.js Cliente Axios + interceptor JWT automatico
│   │       ├── auth.jsx Context de autenticacion global
│   │       ├── constants.jsx Constantes de estados, roles y colores
│   │       └── pages/ Una carpeta/archivo por pagina
│   ├── components/ Componentes reutilizables + shadcn/ui
│   ├── visualization/ Modulo de visualizacion 3D y heatmaps
│   └── electron/ (Generado en proceso de empaquetado)
│       ├── main.js Proceso principal Electron
│       └── package.json Config Electron Builder
├── memory/
│   └── PRD.md Product Requirements Document
├── docs/ Documentacion adicional
├── ARCHITECTURE_DIAGRAM.md
├── IMPLEMENTATION_GUIDE.md
├── QUICK_START.md
└── README.md
```

4 Configuración del entorno de desarrollo

Herramienta	Versión mínima	Notas
Python	3.11+	pyenv o instalar desde python.org
Node.js	18+	nvm recomendado para gestión de versiones
npm	9+	Incluido con Node.js
MongoDB	Atlas cloud o 6+ local	Local: puerto 27017
Git	Reciente	Para clonar el repositorio

```
# 1. Clonar el repositorio
git clone https://github.com/tallafetlopez/kentia_calibration.git
cd kentia_calibration

# 2. Dependencias backend
pip install -r backend/requirements.txt

# 3. Dependencias frontend
npm install --prefix frontend
```

Crear los ficheros .env (no incluidos en el repo)

```
# backend/.env
MONGO_URL=mongodb://localhost:27017/
DB_NAME=calibrationdb
JWT_SECRET=supersecretkey123_herko_calibration_2026
CORS_ORIGINS=http://localhost:3000

# frontend/.env
REACT_APP_BACKEND_URL=http://localhost:8000
```

■ ATENCIÓN

JWT_SECRET debe tener al menos 32 bytes. El valor de ejemplo cumple ese requisito.

5 Variables de entorno

Backend — backend/.env

Variable	Tipo	Descripción	Valor por defecto
MONGO_URL	String	Cadena de conexión a MongoDB. Soporta local y Atlas SRV.	mongodb://localhost:27017/
DB_NAME	String	Nombre de la base de datos MongoDB.	calibrationdb
JWT_SECRET	String	Clave de firma JWT. Mínimo 32 bytes.	supersecretkey123_herko_calibration_2026
CORS_ORIGINS	String	Origen CORS permitido. En producción: localhost:8000.	http://localhost:3000

Frontend — frontend/.env

Variable	Descripción	Valor
REACT_APP_BACKEND_URL	URL base de la API FastAPI. Prefijo de todas las llamadas Axios.	http://localhost:8000

■ INFORMACIÓN

Para el build de producción (instalador .exe), CORS_ORIGINS debe ser http://localhost:8000 ya que React queda servido por FastAPI en el mismo puerto.

6 Arranque en local

Primera ejecución

```
cd backend
python seed.py # Pueba la BD con datos demo
```

Ejecución diaria — 2 terminales

Terminal 1 — Backend:

```
cd backend
python -m uvicorn server:app --reload --host 0.0.0.0 --port 8000
```

Terminal 2 — Frontend:

```
npm start --prefix frontend
```

Servicio	URL
Frontend React	http://localhost:3000
Backend FastAPI	http://localhost:8000
Swagger UI (docs interactivos)	http://localhost:8000/docs
ReDoc	http://localhost:8000/redoc

Diagnóstico y utilidades

```
cd backend
python check_db.py # Verifica conexion a MongoDB Atlas
python -m pytest tests/ -v # Tests de integracion de la API
python seed_users_simple.py # Resetea solo usuarios (si el login falla)
```

■ NOTA TÉCNICA

El backend debe arrancarse siempre desde `cd backend/` — no desde la raíz — para que los imports relativos de `models.py` y `auth_utils.py` funcionen correctamente.

7 Modelos de datos — MongoDB

Colección: users

Campo	Tipo	Descripción
_id	ObjectId	Identificador único generado por MongoDB.
email	String	Email único. Usado como username para el login.
hashed_password	String	Hash bcrypt de la contraseña.
roles	Array[String]	Lista de roles asignados. Puede ser múltiple.
active_role	String	Rol activo actualmente seleccionado.
is_active	Boolean	Si False el usuario no puede iniciar sesión.
created_at	DateTime	Timestamp de creación de la cuenta.

Colección: software_releases

Campo	Tipo	Descripción
_id	ObjectId	Identificador único.
name	String	Nombre de la versión (ej: HKSW_0A_03_102_00).
description	String	Descripción del contenido del release.
a2l_filename	String	Nombre del fichero A2L de referencia.
dbc_filename	String	Nombre del fichero DBC de referencia.
dtc_filename	String	Nombre del fichero DTC (puede ser nulo).
created_by	String	Email del usuario que creó el release.
created_at	DateTime	Timestamp de creación.

Colección: datasets

Campo	Tipo	Descripción
_id	ObjectId	Identificador único.

Campo	Tipo	Descripción
software_release_id	ObjectId	FK al release padre.
name	String	Nombre del dataset.
status	String	Estado: EDIT UNDER_APPROVAL APPROVED RELEASE_CANDIDATE RELEASED DEPRECATED.
scope	String	Ámbito de aplicación.
created_by	String	Usuario que creó el dataset.
created_at	DateTime	Timestamp de creación.
updated_at	DateTime	Timestamp de última modificación.

Colección: labels

Campo	Tipo	Descripción
_id	ObjectId	Identificador único.
dataset_id	ObjectId	FK al dataset padre.
name	String	Nombre del parámetro en el fichero A2L.
value	Any	Valor de calibración actual.
unit	String	Unidad de medida según A2L.
lower_limit	Float	Límite inferior definido en A2L.
upper_limit	Float	Límite superior definido en A2L.
confidence	String	Nivel: EMPTY CALIBRATED VALIDATED DOCUMENTED.
filename	String	Fichero de origen dentro del A2L.
updated_by	String	Email del último usuario que modificó el parámetro.
updated_at	DateTime	Timestamp de última modificación.

8 API REST — Referencia de endpoints

■ INFORMACIÓN

Documentación interactiva en <http://localhost:8000/docs> (Swagger UI). Todos los endpoints /api/ requieren: Authorization: Bearer {token} salvo /api/auth/login y /api/auth/register.

Autenticación

Método	Ruta	Auth	Descripción
POST	/api/auth/login	No	Login. Body: {email, password}. Responde con access_token.
POST	/api/auth/register	No	Registro de nuevo usuario.
GET	/api/auth/me	JWT	Datos y rol activo del usuario autenticado.
POST	/api/auth/switch-role	JWT	Cambia el rol activo. Body: {role: string}.

Software Releases

Método	Ruta	Auth	Descripción
GET	/api/software-releases	JWT	Lista todos los releases con filtros opcionales.
POST	/api/software-releases	JWT	Crea un nuevo release.
GET	/api/software-releases/{id}	JWT	Detalle completo de un release.
PATCH	/api/software-releases/{id}	JWT	Actualiza campos de un release.
DELETE	/api/software-releases/{id}	JWT	Elimina un release (solo admin).
POST	/api/software-releases/{id}/merge	JWT	Merge de datasets. Body: {base_id, overlay_id, strategy}.

Datasets

Método	Ruta	Auth	Descripción
GET	/api/datasets	JWT	Lista datasets. Filtros: release_id, status, scope.
POST	/api/datasets	JWT	Crea dataset vinculado a un release.
GET	/api/datasets/{id}	JWT	Detalle con todos sus labels.
PATCH	/api/datasets/{id}	JWT	Actualiza metadatos del dataset.
POST	/api/datasets/{id}/submit	JWT	Envía a UNDER_APPROVAL.
POST	/api/datasets/{id}/approve	JWT	Aprueba (requiere rol aprobador).
POST	/api/datasets/{id}/reject	JWT	Rechaza. Vuelve a EDIT.
POST	/api/datasets/{id}/release	JWT	Promueve a RELEASE_CANDIDATE.
POST	/api/datasets/{id}/publish	JWT	Publica el dataset (RELEASED).
POST	/api/datasets/{id}/deprecate	JWT	Marca como DEPRECATED.
GET	/api/datasets/{id}/export-dcm	JWT	Exporta fichero .dcm DAMOS 2.0.

Labels, Reviews, Vehicles, Audit y Admin

Método	Ruta	Auth	Descripción
GET	/api/datasets/{id}/labels	JWT	Lista labels con filtros: confidence, filename, search.
PATCH	/api/datasets/{id}/labels/{lid}	JWT	Actualiza un label. Body: {value, confidence}.
GET	/api/reviews	JWT	Reviews pendientes para el rol activo.
POST	/api/reviews/{id}	JWT	Aprobar o rechazar una revisión.
GET	/api/vehicle-sw-ids	JWT	Lista asignaciones VIN↔Software.
POST	/api/vehicle-sw-ids	JWT	Crea nueva asignación.
DELETE	/api/vehicle-sw-ids/{id}	JWT	Elimina una asignación.
GET	/api/audit-log	JWT	Log de auditoría con filtros.



Método	Ruta	Auth	Descripción
GET	/api/users	JWT+Admin	Lista todos los usuarios.
PATCH	/api/users/{id}	JWT+Admin	Actualiza roles o estado activo.
POST	/api/seed	JWT+Admin	Re-ejecuta el seed de datos demo.



9 Autenticación JWT y sistema de roles

La autenticación usa JSON Web Tokens firmados con HS256. El frontend almacena el token en memoria (no en localStorage) y lo inyecta en todas las peticiones mediante el interceptor de Axios.

```
POST /api/auth/login
Body: { "email": "user@herko.dev", "password": "password123" }
Response: { "access_token": "eyJhbGciOi...", "token_type": "bearer" }

# Header en peticiones autenticadas:
Authorization: Bearer eyJhbGciOi...
```

Payload JWT

```
{
  "sub": "user@herko.dev",
  "roles": ["Calibration_Engineer", "PI_Engineering_Manager"],
  "active_role": "Calibration_Engineer",
  "exp": 1748000000
}
```

auth_utils.py — funciones clave

```
def create_access_token(data: dict) -> str:
    return jwt.encode(data, JWT_SECRET, algorithm="HS256")

def get_current_user(token: str = Depends(oauth2_scheme)) -> dict:
    payload = jwt.decode(token, JWT_SECRET, algorithms=["HS256"])
    return payload
```

Roles del sistema

Nombre en la UI	Código en sistema	Capacidades clave
Project Manager	PD_Project_Manager	Crear/editar releases. Promover datasets. Gestión completa.
Calibration Engineer	Calibration_Engineer	Editar labels. Enviar datasets a revisión.
PI Engineering Manager	PI_Engineering_Manager	Aprobar/rechazar (ingeniería).
PI Regulatory Compliance	PI_Regulatory_Compliance_Specialist	Aprobar/rechazar (regulatorio).

Nombre en la UI	Código en sistema	Capacidades clave
V&V; Engineer	PD_Verification_Validation_Engineer	Validar parámetros. Promover a Release Candidate.
Configuration Manager	Configuration_Manager	Gestión de configuración y versiones.
DM Administrator	DM_Administrator	Admin completo: usuarios, roles, seed.
Post Sales Engineer	Post_Sales_Engineer	Asignación de software RELEASED a vehículos por VIN.

■ NOTA TÉCNICA

Un usuario puede tener múltiples roles (campo roles: Array[String]). active_role determina los permisos activos. /api/auth/switch-role actualiza active_role en el token regenerado.

10

Build y empaquetado del instalador .exe

1

Cambiar CORS en backend/.env

```
Cambiar CORS_ORIGINS=http://localhost:3000 a http://localhost:8000
```

2

Compilar frontend (desde raíz)

```
npm run build --prefix frontend  
# Verifica: frontend/build/index.html
```

3

StaticFiles en backend/server.py (al final, tras todas las rutas /api/)

```
import sys, os  
from fastapi.staticfiles import StaticFiles  
_base = getattr(sys, '_MEIPASS',  
os.path.dirname(os.path.abspath(__file__)))  
_build = os.path.join(_base, 'build')  
if os.path.isdir(_build):  
app.mount('/', StaticFiles(directory=_build, html=True),  
name='static')
```

4

PyInstaller (desde backend/)

```
pip install pyinstaller  
pyinstaller --onefile --name CalibrationManagerServer \  
--add-data ".env;." \  
--add-data "../frontend/build;build" \  
--hidden-import motor \  
--hidden-import jose \  
server.py  
# Resultado: backend/dist/CalibrationManagerServer.exe
```

5

electron/main.js

```
# Arranca CalibrationManagerServer.exe como proceso hijo  
# Polling localhost:8000 cada 500ms, timeout 30s  
# BrowserWindow: 1400x900, autoHideMenuBar:true  
# Al cerrar ventana: mata proceso servidor limpiamente
```

6

Compilar Electron (desde electron/)

```
npm install && npm run build  
# Resultado: electron/dist/win-unpacked/CalibrationManager.exe
```

7

Inno Setup — generar instalador

```
winget install JRSoftware.InnoSetup # si no está instalado  
ISCC.exe calibration_manager_installer.iss  
# Resultado: installer_output/CalibrationManager_Setup.exe
```

■ NOTA TÉCNICA

El instalador generado debe pesar más de 120 MB (Electron ~100 MB + Python/FastAPI ~30-50 MB + frontend ~5 MB). Si pesa menos de 50 MB, PyInstaller no incluyó todas las dependencias.

11 Bugs conocidos y sus soluciones

#1 — backend/.env no existía

Archivo: backend/.env

```
Crear con: MONGO_URL, DB_NAME, JWT_SECRET, CORS_ORIGINS
```

#2 — ModuleNotFoundError: No module named 'models'

Archivo: backend/server.py

```
Arrancar siempre desde cd backend/, no desde la raíz del proyecto.
```

#3 — CORS bloqueado: allow_origins=['*'] + allow_credentials=True

Archivo: backend/server.py

```
Combinacion wildcard + credentials bloqueada por browsers.  
Usar CORS_ORIGINS=http://localhost:3000 (o :8000 en prod)
```

#4 — numpy/scipy incompatibles

Archivo: requirements.txt

```
pip uninstall -y numpy scipy  
pip install 'numpy>=1.21.6,<1.28.0' scipy --force-reinstall
```

#5 — date-fns v4 incompatible con react-day-picker

Archivo: frontend/package.json

```
Degradar a date-fns version ^3.6.0
```

#6 — React v19 incompatible con react-day-picker

Archivo: frontend/package.json

```
Degradar react y react-dom a version ^18.3.1
```

#7 — JWT_SECRET menor de 32 bytes

Archivo: backend/.env

```
Clave debe tener >=32 bytes.  
Correcto: supersecretkey123_herko_calibration_2026
```

#8 — useEffect con dependencia 'load' faltante

Archivo: DatasetDetailPage.jsx, SoftwareReleaseDetailPage.jsx, SoftwareReleasesPage.jsx

```
Envolver 'load' en useCallback con dependencias correctas
```

12

Registro de cambios (Changelog)**v1.0.0 — Mayo 2026 — Release inicial — Kentia Engineering**

- Workflow completo EDIT → UNDER_APPROVAL → APPROVED → RELEASE_CANDIDATE → RELEASED → DEPRECATED.
- Importación de ficheros A2L con extracción automática de parámetros.
- Export DCM en formato DAMOS 2.0 filtrable por scope, maturity y filename.
- Merge de calibraciones con preview de conflictos y tres estrategias.
- Warning banner para parámetros sin DCM con modal de límites A2L.
- Role switcher en navbar para cambio de rol sin logout.
- Admin Panel con gestión de usuarios y asignación de roles por checkboxes.
- Vehicle Assignment con asignación VIN ↔ Software Release.
- Audit Log completo con todos los cambios del sistema.
- Autenticación JWT HS256 con hash bcrypt.
- Módulo de visualización 3D / heatmaps (visualization/).
- Corrección de 8 bugs detectados en sesión de revisión del 03/05/2026.

Calibration Manager v1.0.0 — Kentia Engineering — Documentación Técnica — Mayo 2026